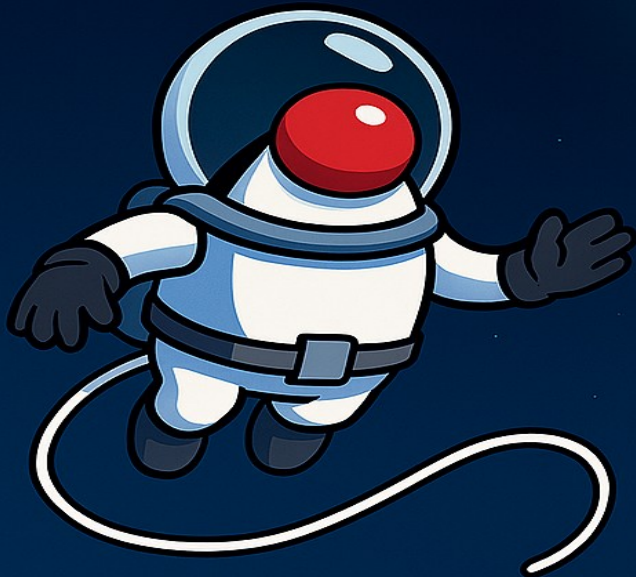
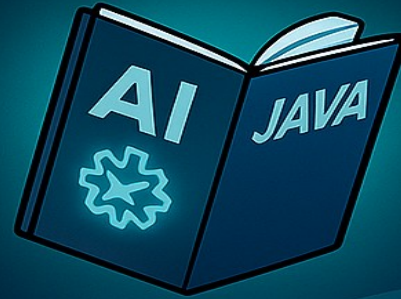


A Hitchhiker's Guide to the AI Galaxy



For Java Developers
By Vishal Shanbhag

Who Am I

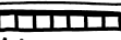
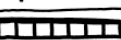
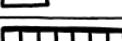
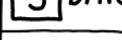
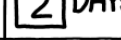
- www.linkedin.com/in/vishal-shanbhag-70b679a
- <https://medium.com/@vvsvish>
- <https://javapro.io/author/vishalshanbhag/>
- <https://www.baeldung.com/author/vishalshanbhag>



```
class Galaxy(alculator {  
  {  
    public static Void main(String[] args);  
    //-taking data result >>  
    modid inr passe! - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. .." # result());  
  }  
}
```



HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE
EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	 4 WEEKS	 3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	 8 WEEKS	 6 DAYS	 1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	 4 WEEKS	 6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	 5 WEEKS	 5 DAYS	 1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	 10 DAYS	 2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	 2 WEEKS	 1 DAY
	 1 DAY					 8 WEEKS	 5 DAYS

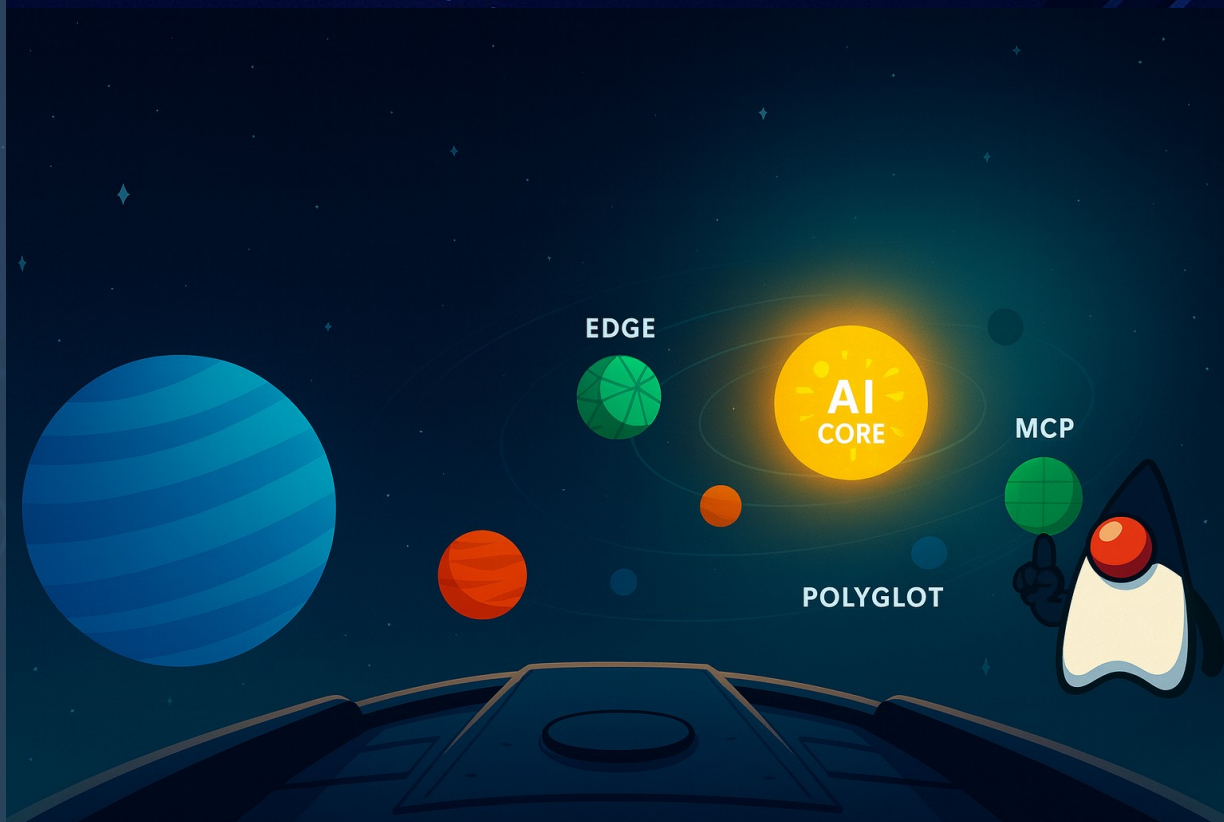
Edge – Demo Time

- Java FX UI
- LM Studio
- Ollama CLI
- Quarkus Backend

```
class Galaxy(calculator {  
    {  
        public static Void main(String[] args);  
        // -lrm:ing data result >>  
        modid inr passe| - calculateanswer();  
  
        System.out.println("The answer to life,  
        the universe. .." # result());  
    }  
}
```

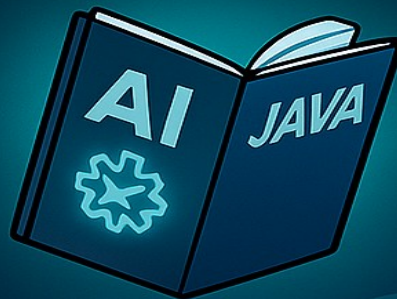
The Galaxy Map

- Why Generative AI + Java
- Three worlds:
 - Edge
 - Polyglot
 - Protocol (MCP)
- Other Noteworthy Planets
 - Cloud LLM Providers (e.g. Azure)
 - Vector Stores (e.g. Milvus, Qdrant, Pinecone)
 - Model Training (DJL, Tensor Flow)
 - Many Other Libraries



Why GenAI + Java is a Formidable Combination

- Battle-tested: decades of enterprise trust
- Type-safe, object-oriented, mature ecosystem
- “Write once, run anywhere” still holds
- Excellent tooling, frameworks, and developer base



- Historically: weak AI/ML adoption vs Python
- GenAI changed the game — Java is catching up fast

Jargon Bingo

- **LLM** = Large Language Model
- **Inference** = Fancy Word for Auto Complete
- **Hallucination** = AI lies and does not know that
- **Prompt** = Fancy word for instructions given to LLM
- **Context** = Information shared with LLM in each API call.
- **Tool** = Custom Functions LLM can ask to be executed
- **Agent** = Automation with Decision making handed over to LLM
- **RAG** = Retrieval Augmented Generation = Telling AI to look outside its memory data.

```
class Galaxy(calculator {  
    public static void main(String[] args){  
        // -llm:ing data result >>  
        System.out.println("The answer to life,  
        the universe. ..." # result());  
    }  
}
```

Planet Edge

Today's Tool Stack

- Ollama
- LM Studio
- Java + Quarkus + Langchain4j
- Azure AI Foundry

Other Noteworthy Libraries:

- DJL (Deep Java Library)
- ONNX Runtime
- TensorFlow Java bindings

```
class Galaxy(calculator {  
  {  
    public static Void main(String[] args);  
    // -lrmguing data result >>  
    modid inr passe] - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. ..." # result());  
  }  
}
```

Note: Local inference, enterprise privacy, no API keys.

Planet Edge – Key Dependencies

```
<dependency>
```

```
  <groupId>io.quarkiverse.langchain4j</groupId>
```

```
  <artifactId>quarkus-langchain4j-ollama</artifactId>
```

```
</dependency>
```



```
class Galaxy(alculator {  
  {  
    public static Void main(String[] args);  
    // -llm:ing data result >>  
    modid inr passe] - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. .." # result());  
  }  
}
```



Planet Edge - Service

```
import dev.langchain4j.service.UserMessage;
import io.quarkiverse.langchain4j.RegisterAiService;
import jakarta.enterprise.context.ApplicationScoped;

@RegisterAiService
@ApplicationScoped

public interface EdgeLLMService {
    String infer(@UserMessage String prompt);
}
```

Planet Edge - Config

```
quarkus.langchain4j.timeout=180s
# Ollama Config
quarkus.langchain4j.chat-model.provider=ollama
quarkus.langchain4j.ollama.base-url=http://localhost:11434

# Chat model
quarkus.langchain4j.ollama.chat-model.model-name=llama3.1:8b
quarkus.langchain4j.ollama.chat-model.temperature=0.2
quarkus.langchain4j.ollama.log-requests=true
```

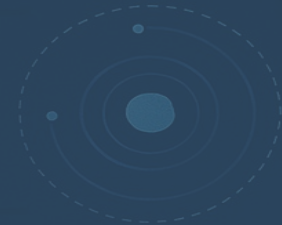

Planet Edge - Alternate Config

```
# Azure OpenAI Config
```

```
quarkus.langchain4j.azure-openai.api-key=${AZURE_OPENAI_API_KEY}  
quarkus.langchain4j.azure-openai.endpoint=${AZURE_OPENAI_ENDPOINT}  
quarkus.langchain4j.azure-openai.temperature=0.2  
quarkus.langchain4j.azure-openai.api-version=2024-02-01  
quarkus.langchain4j.azure-openai.log-requests=true
```



```
class Galaxy(alculator {  
  {  
    public static Void main(String[] args);  
    // -lrg:ing data result >>  
    modid inr passe] - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. .." # result());  
  }  
}
```



Planet Edge - Lessons

- Model Runtime Size Matters
- More GPU Memory = Speed
- Lower Temperature = Accuracy
- LLM has anterograde amnesia
- Context Length Matters



```
class
{
    pub
    // - calculate the answer to the question
    modid inr passe] - calculateanswer();

    System.out.println("The answer to life,
    the universe. ..." # resul[]);
}
```

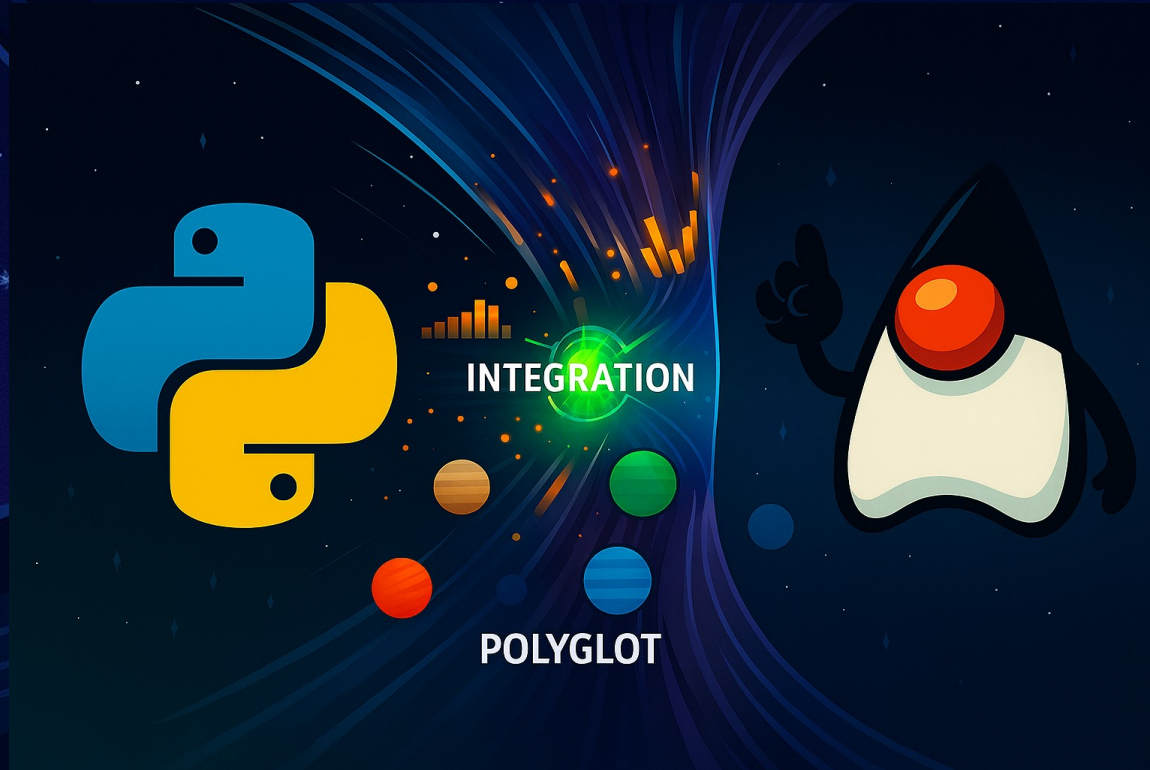

Planet Edge – Handling Context

Topics to Explore

- Conversation History
- RAG
- Vector Databases
- Remember HTTP – Statelessness

```
class Galaxy(alculator {  
  {  
    public static Void main(String[] args);  
    // -lrg:ing data result >>  
    void in:pasel = calculateanswer();  
    System.out.println("the universe. .." # result());  
  }  
}
```


Planet Polyglot



- GraalVM
- Truffle framework
- Python interoperability (GraalPy)

```
n(string[] args)  
>  
calculateAnswer();  
  
answer to life,  
result();
```

Note: Extend Java with Python's AI ecosystem without leaving the JVM.

Planet Polyglot – Advantages

- Access Python's mature AI/ML ecosystem directly from Java
- Train Models using Python, Deploy Applications using Java
- Share memory efficiently via GraalVM no HTTP bridges
- Consistent packaging/deployment compared to mixed-language microservices
- Lower operational overhead
- Ability to reuse existing enterprise JVM infrastructure
- Pairs well with cloud-hosted LLMs: local Python for data prep, Java for scalable deployment.

```
class GalaxyCalculator {  
    // -llg-ung data result >>  
    modid inr passe| - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. ..." # result);  
}
```


Planet Polyglot – Demo Time

- GraalPy
- CLI test case



```
class Galaxy(alculator {  
{  
    public static Void main(String[] args);  
    // -lirg:ing data result >>  
    modid inr passe| - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. .." # result());  
}
```



Planet Polyglot – Lessons Learned

- Warm-up time matters: GraalVM polyglot context creation isn't free
- Data marshalling between languages must be designed intentionally
- Some Python libraries assume CPython quirks → not always GraalPy-compatible
- Debugging polyglot code requires new muscle memory (stack traces cross languages)
- Version mismatches between Python packages can cascade fast
- You still need Python literacy; polyglot isn't a magic "run Python without knowing Python" trick
- Don't overuse it — Java for structure, Python for the AI bits only

```
class Galaxy(calculator {  
    public static Void main(String[] args);  
    // -lir-ung data result >>  
    modid inr passe] - calculateanswer();  
    return null; // in the answer to life,  
    the universe. ... # result();  
}
```

From Local Dev to Cloud Deployment

- Prototype locally with Ollama + LangChain4j
- Swap to Azure AI Foundry with one provider change
- Use GraalVM native-image for serverless Java functions
- Fast startup → ideal for event-driven AI workloads



```
class Galaxy(allocator {  
    public static void main(String[] args);  
    //-llg-llg data result >>  
    modid inr passe] - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. .." # result());  
}
```


Planet Protocol – MCP

- Thank you **Anthropic**
- Model Context Protocol = Fancy Word for AI can make API calls.
- Think Swagger / Open API but meant for LLMs

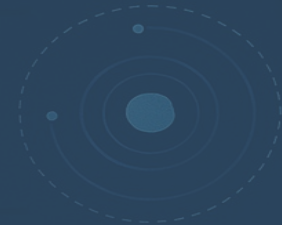
```
class Galaxy {  
    public static void main(String[] args);  
    // -lrg-ung data result >>  
    modid inr passe| - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. ..." # result());  
}
```


Planet Protocol – Demo Time

- From JavaFX UI



```
class Galaxy(alculator {  
{  
    public static Void main(string[] args);  
    //-loading data result >>  
    modid inr passe| - calculateanswer();  
  
    System.out.println("The answer to life,  
    the universe. ..." # result());  
}
```



Planet Protocol – Code

```
// Tool Definition on MCP server

@Tool(description="Returns the sentiment score of a given text")
public String score(

    @ToolArg(description = "Text whose sentiment is to be
calculated") String text) {

    // Method body

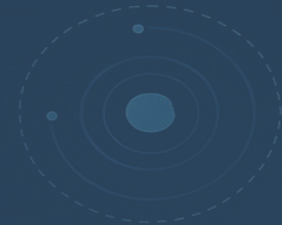
}
```



```
class Galaxy(alculator {
{
    public static Void muin(string[] args);
    //-llrg-ung data result >>
    modid inr passe| - calculateanswer();

    System.out,pfin()n(The answer to life,
the universe. .." # resul{};

}
```



Planet Protocol – Code

```
# MCP Toolbox Config
# Ensure that the MCP is running before enabling these settings
quarkus.langchain4j.mcp.default.transport-type=http
quarkus.langchain4j.mcp.default.url=http://localhost:9000/mcp/sse
```



```
class Galaxy(alculator {
{
    public static Void main(String[] args);
    // -langchain data result >>
    modid inr passe] - calculateanswer();

    System.out.println("The answer to life,
    the universe. .." # result());
}
```



Planet Protocol – Code

```
// Somewhere inside Interface annotated with @RegisterAiService

@SystemMessage(""" You are an AI assistant that helps users and you have
access to various tools to assist in answering questions. When using a
tool, always convert the tool's response into a natural, human-readable
answer. If the user's question is unclear, politely ask for clarification.
It's ok to use tools to answer questions if needed otherwise, answer it
directly using your own knowledge. Always communicate in a friendly and
professional manner, and ensure your responses are easy to understand.
""")
@McpToolBox("default")
String chatWithTools(@MemoryId String chatId, @UserMessage String
message);

class Galaxy(calculator {
{
    public String calculateAnswer(@UserMessage String chatId, @UserMessage String
    // -> long data result >>
    modid inr passe] - calculateanswer();

    System.out.println("The answer to life,
    the universe. ..." # result);
}
}
```

Planet Protocol – Lessons Learned

- Orchestration frameworks introduce their own complexity—keep flows simple
- Chat-style LLM workflows behave non-deterministically → handle errors defensively
- Clear boundaries are crucial: “What’s Java’s job? What’s the model’s job?”
- Observability must be added manually — logs, traces, input/output recording
- Security is not optional: sanitize tool outputs, enforce access controls, validate prompts
- Start small: integrate one Java service as a tool before exposing your entire domain model

```
class Galaxy(calculator {  
    {  
        public static void main(String[] args)  
        {  
            // ...  
            modid inr passe] = calculateanswer();  
  
            System.out.println("The answer to life,  
            the universe. ..." # result);  
        }  
    }  
}
```


Cloud Ready Path Forward

- Replace local LLM with Azure AI Foundry
- Deploy Java services as GraalVM native images
- Use LangChain4j for consistent API surface
- Keep your existing Java skills—just add AI

```
public static void main(String[] args);  
// -lrg:vm.native-image --native-image --native-image --native-image  
System.out.println("The answer to life,  
the universe. ..." # result());  
}
```


Q & A

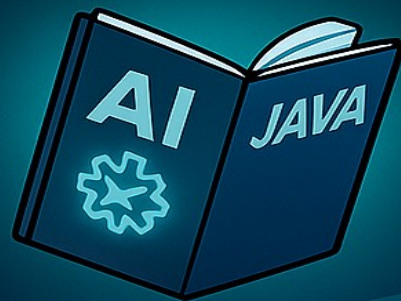
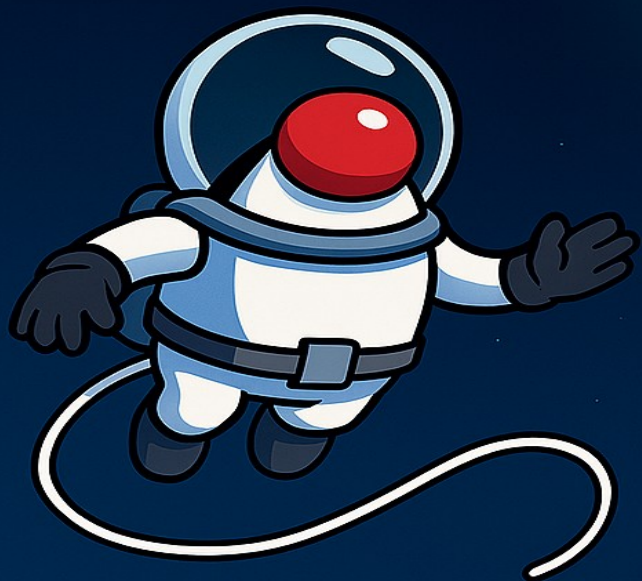
- Where is the code?

<https://github.com/vshanbha/Duke42>

- Slides? – Will be shared

- Anything Else?

```
class Galaxy(alculator {  
    {  
        public static Void main(String[] args);  
        // -lrg-ung data result >>  
        modid inr passe! - calculateanswer();  
  
        System.out.println("The answer to life,  
        the universe. .." # result());  
    }  
}
```



Thank You